

```

class Transmission:

    def __init__(self, trame):

        self._id = None

        self._temperature = None

        self._humidite = None

        self._trame = trame

        self.decoder()

    def __repr__(self):

        """ Méthode magique pour affichage """

        return f"ID : {self._id} / Temp. : {self._temperature}°C / Hum. : {self._humidite}%"

    def decoder(self):

        self.decoder_id()

        self.decoder_temperature()

        self.decoder_humidite()

    def decoder_id(self):

        self._id = int(self._trame[0:8], 2) # int(s, 2) : conversion binaire -> décimal

    def decoder_temperature(self):

        self._temperature = (int(self._trame[16:28], 2)-900)/10

    def decoder_humidite(self):

        self._humidite = int(f'{int(self._trame[28:32], 2)}{int(self._trame[32:36], 2)}')

    def get_id(self):

        return self._id

    def get_temperature(self):

        return self._temperature

    def get_humidite(self):

        return self._humidite

    def est_valide(self):

        if len(self._trame) != 40: # Question 4

            return False

```

```
if self._trame[0:8].count('1') % 2 != int(self._trame[36]):
    return False

if self._trame[8:16].count('1') % 2 != int(self._trame[37]):
    return False

if self._trame[16:28].count('1') % 2 != int(self._trame[38]):
    return False

if self._trame[28:36].count('1') % 2 != int(self._trame[39]):
    return False

return True

#TEST#

test = Transmission('0010101011001000010010001100011000101101')

print(test)

print(test.est_valide())

# >>> ID : 42 / Temp. : 26.4°C / Hum. : 62%

# >>> True

#le code du fichier analyse.py retourne une erreur

# car les trames dans data.txt ne sont pas tout le temps

# correctes : certaines font moins ou plus de 40 caractères

# pour corriger cela, on modifie la méthode est_valide pour

# qu'elle vérifie la taille de la trame (réponse question 4)
```